

Soft^{ware} Engineering (S.E.)

< BLOCK 1: Agile Software development >

1. 定义

Def 软件工程 An engineering discipline that is concerned with all aspects of software production.
• Software E. is a layered technology (Tool → Method → Process → Quality focus) (SE)

2. Software Process

Def Software process A software process is a set of structured activities that leads to the production of the software.
更细节的, 贯穿本课程的:
Requirement specification → Development
↓
Evolution ← { Test Deployment }
Analysis (create a conceptual model)
Design (Implementable model)
Implementation (实现)

Software process model:

- ① Waterfall model (Sequential approach, 慢, 文档多)
- ② Evolutionary development (快, 多 versions)
- ③ The Rational Unified Process (RUP) ~~快~~
- ④ Agile Software Development

3. 简单介绍 Agile Software Development (Rapid development and delivery)

Def Agile process The processes of specification, design, implementation and testing are concurrent, referred to as an iteration.

① 关于 Agile.

A set of best practices in SE. || Focus on code. || iterative approach
Visible process || Extreme Programming || Test Driven Development (TDD)
pair programming

还有 PPT-样, 提供了“替人换位”式学习。哈哈!

4. Requirements

Def Requirement A requirement is a feature that your system must have in order to satisfy the customer
Def Stakeholder Any person or organization who is affected by the system in some way and so who has interest

注意 Requirement 会变, 难交流, 是 SE 中最难的问题, 所以 ↓

① Requirement Engineering (帮工程师理清真正需求)

需求分类 Functional requirements (should do)
Non-Functional requirements (System's properties and constraints)

不同 Requirements 中有 conflicts 是很正常的事。看 stakeholders 的脸色 trade-off.

output Software Requirements Specification (SRS)

What is required of system developers. 应该做什么

② Requirements Capture (Fact-finding)

Background Reading; Interviewing; Observation; Document Analysis.

5. Agile 中的 Requirements

在 Agile 中, Requirement 又是 user stories. 漏洞需求, 任何人都可以写

Def User story User stories are short, simple description of feature. They typically follow a simple template: As a...; I want...; So that...

注意, 开发团队应该把 story break down into tasks.

2) 长的 story 被拆分为 Epics, 在开发中再拆 Epics 拆分为 story.

3) 每个 story 应配备 Acceptance Criteria, 是检验是否完成需求的 test.

Def Product backlog Product backlog, which is a prioritised list of the functionality to be developed in a product or service
Epic story → task

① 在 backlog 中, 需要确定 Prioritisation of stories:

- 一种确定的方式: Dynamic system development method (DSDM).

将故事分为: must have, should have, could have, Want to have

⇒ MoSCoW

(补充) 故事应该: Valuable, Estimable, Small, testable.

② Estimating: 估计关于 work 完成的时间、努力等。

一般方法: T-shirt sizing (相对): Ideal time (天时)

Def
Story
points

Story points: an arbitrary measure that allows the teams to understand the size of the effort.

Team members 有不同能力和经验, 使用精确 time 估计会有较大差异; Arbitrary measure can avoid over-estimating. 相对 T-shirt size, story point 更准确估计工作量的 size of the effort.

③ Prototyping (Physical or logic)

没有 technical 技巧
↑
low time, cost.

1) Low-fidelity (低保真): paper and sketch; Quick, effective

2) Medium-fidelity: 功能有限, 有可点击的交互区域。

3) High-fidelity: 考虑 user interface (UI), user experience (UX) 细节和功能的最终设计相似。

< BLOCK 2: Development & Test >

1. Analysis: 为3 precise understanding of requirement, 关注系统内部。

① Conceptual modelling

↓ ~~识别概念~~ 识别概念

Def
识别概念

Aim to identify the individual concepts which exist within a problem, described using UML class diagrams.

② Analysis Class (三种种类)

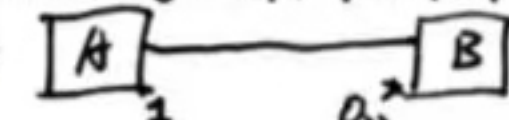
1) Entity class: model information that is long-lived and persistent

2) Boundary class: model the interaction (Button, page 等)。

3) Control class: Deal with performing tasks, getting/setting data and coordinating behaviour.

③ Class Relationships (两种关系)

1) Association: 类之间的 bidirectional connection (数据有双向流动)

用 multiplicity indicators 表示连接数量: 

0..2 (0到2), 0..* (0或更多)

(1个A对应0或更多B)

2) Inheritance

subclass will inherit all attributes, methods, relationships defined in any of its superclasses, 有少许的继承。

2. Design & 数据访问

④ Role of Design.

Design transforms the analysis model into a design model that serves as a blueprint for software construction.

② Encapsulation: 防止直接访问, 设置 set/get 方法。

③ Modularity: Separate the functionality of a program into independent, interchangeable modules.

④ Coupling (耦合): metric of dependencies between subsystems. } Tight ↑ 开发
Loose ↓ 维护

⑤ Cohesion (内聚): metric of dependencies within a subsystem } High ✓
Low

⑥ Refactoring (重构): intended to improve nonfunctional attributes of the software.

⑦ OOA 或 OO (Object-Oriented): Easier maintenance, class reusable.

3. Implementation.

goal: mapping Design to code, implement the system in terms of components.

① Build

Def
Build

A "build" is an executable version of the system, normally a part of the system.

Def
integration
build plan

An integration build plan describes the sequence of builds required in an iteration.

② 再谈 Association

JAVA 没有双向连接, 只有 unidirectional link.

1 → 1: 强耦合, (ex: A → B, A 中有 B, B 不可操作 A).

1 → n: 仍然强耦合, 但使用 Collections.

4. Testing

① 目的 } Validation testing: software meets its requirements } Build
Defect testing: Discover defects } Confidence

② Testing process

Stakeholders to confirm the system meet the business needs

unit testing ↔ system testing ↔ Acceptance testing

在类似真实环境中 Test 整个应用。
test individual component such as a function or a method.

<补充> 为什么在TDD unit test中, Refactoring很重要?
Answer: TDD ^{开发} code in small independent increments. 它有助于减少 code duplication and poorly structured code.
 Regular refactoring can improve the code structure s.t. it's more readable and easy to change

③ Testing: Techniques

- 1) Black-Box testing (test functional requirements)
 - Partition testing (选择不同参数数据对function Fed input)
 - Regression Testing (每个Build时测试全部逻辑, 确保新添加功能不影响)
- 2) White-Box testing (test internal program logic)
 - Basis Path Testing
 - Cyclomatic Complexity: $\# \text{ 决策} (\diamond) + 1$
 - Number of test (路径复杂以后再随便造路径)
 - 使得每条语句执行至少一次
 - Mutation Testing
 - make small change to source code
 - $if(x=0) \rightarrow if(x!=0)$
 - if test fails, the test is robust.

5. 简单介绍 Test Driven Development (TDD)

TDD: write tests prior to write the production code.
 JUnit: 支持TDD的 unit test 框架 (自动的).
 使用 `@Test` 表明 test 方法, 系统 main 自动执行 (公共, 无参, 无返回值)
 使用 `assertEquals` ("期望", 实际) 来辅助测试.

< BLOG 3: Architecture & Project Management & Design Principles >

1. Software Architecture

Def ① A Software Architecture is a description of how a Software System is organized
 一般用UML描述

- 注
- 1) Non-functional requirements 是否架构包含的, Requirement 驱动 Architecture
 - 2) 作用: System analysis, reuse, Communication
 - 3) Agility 中的 architecture, 越早着手 architecture 越好, 因为不灵活.

② Architectural patterns

Def Architectural patterns are a means of representing, sharing and reuse ~~pattern~~ knowledge. 这为快速安全开发.

- 1) Web-based architectures
- 2) Distributed System architecture

Web-based 的单点连接表现不好, 可用一些 techniques 优化

- Load balancing: The distribution of tasks over a set of Resources with aim of making their overall processing more efficient.
- Cloud computing: delivery of on-demand computing services typically over the internet.
- Caching

3) RESTful architecture

Representational State transfer (REST) 有 6 个 Constraints

Client-Server, Cacheability, Uniform interface
 Statelessness, Layered system, Code-On-Demand

4) Mobile application architecture

通常指 layered architecture, 可
 presentation $\rightarrow$ Business $\rightarrow$ Data

2. Project Management.

AIM Ensure the Software is delivered On-time, ~~with~~ Within budget, and With Quality. 时间, 预算, 质量.

① Software Projects' Distinctions (特性)

- 1) Intangible (Can not see)
- 2) flexible
- 3) Not standardised
- 4) many software projects are "One-off" Project.

Technologies are change to fast
 Old experiences may be obsolete.

② Project management activities

1) Project planning

— 考虑约束 Constraints, parameters, 建立 milestones, schedule
并不断 iteration 的过程, 可从 Quantity, Validation 等多个角度 plan

Def Milestones are the recognisable end-points of a process activity

必须将整个 project broken down into basic activities 并建立 milestones.

Def Project schedule: Estimate time and resource required to complete activities and organise them into a coherent sequence

注: 一般有 N 个要点: Split project into separate tasks, concurrently minimise task dependencies.

<如何表示 Schedules>

(a) Task chart

(b) Activity network } BOA
AON $\Rightarrow$ 详细整理 PPT

(c) Bar chart (schedule against calendar time)

③ Agile project management

敏捷项目管理是 plan-driven. Agile project 是不同形式:

Adapted to incremental development and the particular strengths of agile methods. (Scrum approach)

EX 一个显著的特征: short daily stand up meeting.

purpose: 让所有人知道目前 project 正在干什么, 以及发现问题可以及时沟通解决.

注: share information

share the progress since last meeting
problem and plan

Agile 中, 重点在于 visibility, communication, trust.

<补充> AI 项目管理: Data analysis, Risk Prediction, Optimize resource allocation 特点

3. Design Principles (SOLID).

用高子 code 的复杂度与问题, 才可以用好的 code (UML 是一种方法).

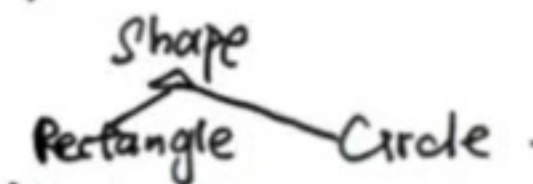
① Single Responsibility Principle (SRP)

Rule: 一个类应该专注于单一功能, 非常直观.

② Open-Closed Principle (OCP)

Rule: "Open for extension, closed for modification".

关键在于使用 Abstraction 类. 例如



只用在子类中修改已有的 draw() 方法,
不必单独有一个类对各种类型处理.

③ Liskov Substitution Principle (LSP)

Rule: 子类可随意替换父类而不引起错误.

注: ① 子类实现父类方法, 且不修改已有实现方法.

② 子类不扩展父类行为, 不可增加新方法, 新属性.

③ 返回值, 参数与父类兼容.

即一种比 "IS-A" 描述更严格的继承规则.

④ Interface Segregation Principle (ISP)

接口应该尽可能小, 以免子类实现 "空" 方法.

即接口小则与具体类需求匹配.

⑤ Dependency Inversion Principle (DIP)

Rule: 高层不依赖低层, 都依赖于抽象;

抽象不依赖细节, 细节依赖于抽象.

高层和低层之间通过抽象层, 低层修改不影响高层.

高层修改也不影响低层. 凡高层直接依赖于低层, 则违背 DIP

⑥ Don't Repeat Yourself (DRY) $\subseteq$ OCP.

将重复代码提取成一个函数, 减少重复.

其本质 OCP 的一种表达.

总结: SRP, OCP, LSP, ISP, DIP (DRY $\subseteq$ OCP).

Solid 设计原则.